

데이터 마이닝 특론

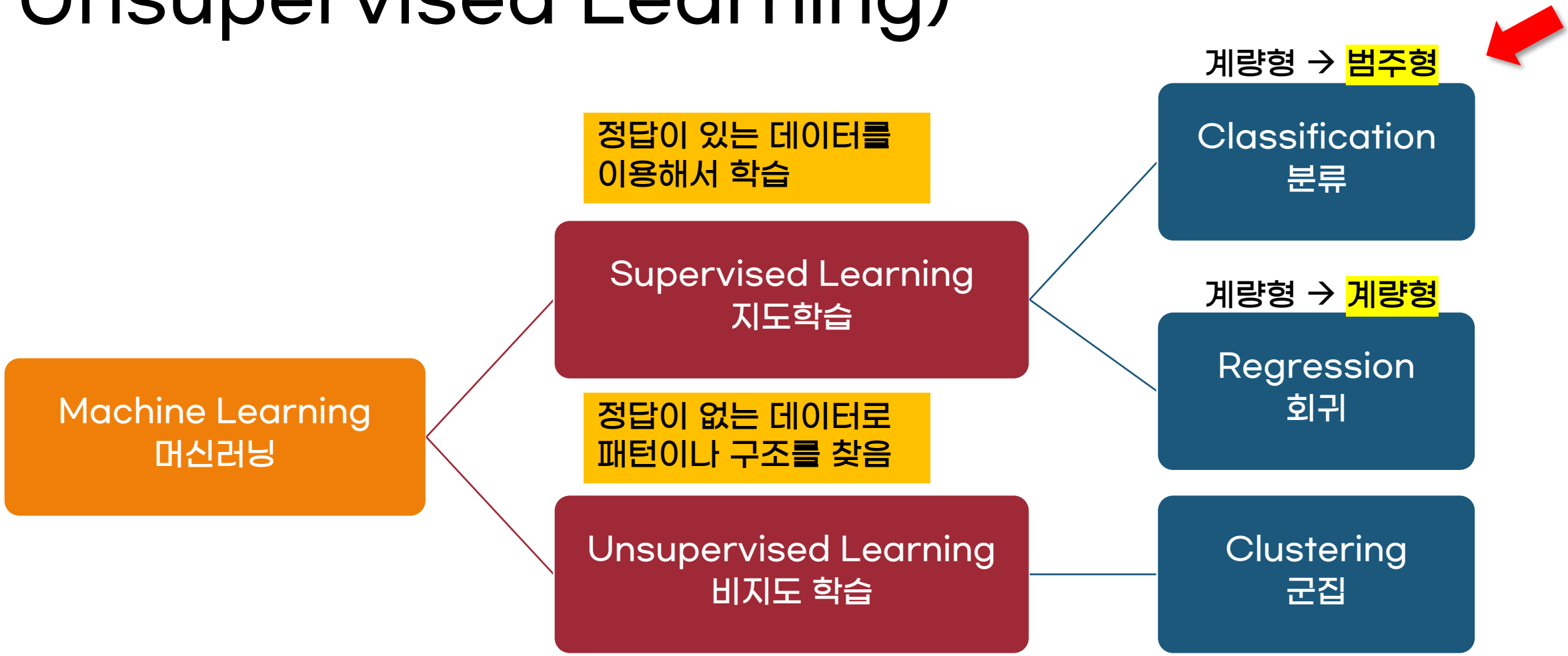
Week 6

서울과학기술대학교
SEOULTECH Seoul National University of Science & Technology

Topics

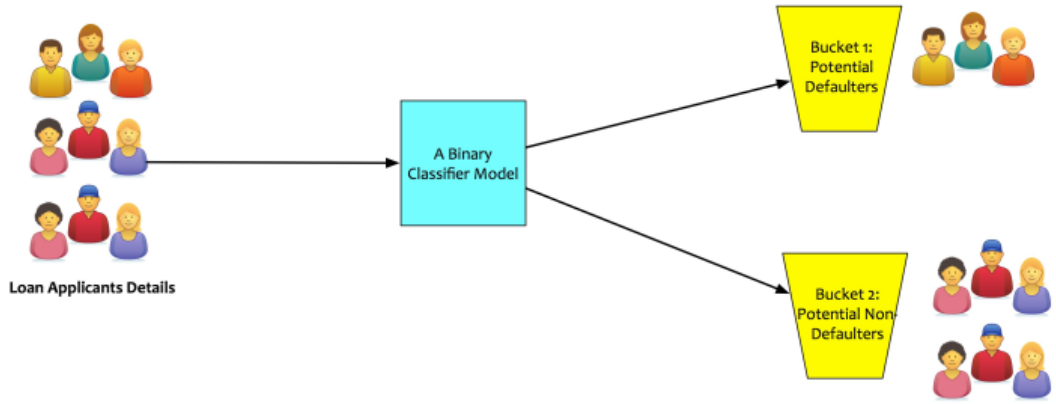
- 지도학습 II: 분류 분석
- 붓꽃 분류모델 실습

지도학습과 비지도학습 (Supervised and Unsupervised Learning)



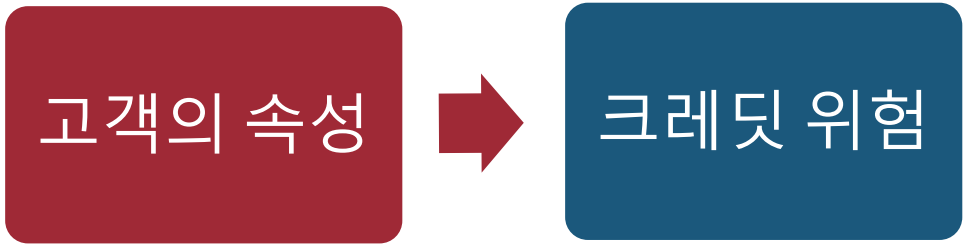
분류 (Classification)

- A machine learning technique that uses known data to determine how the new data should be classified into a set of existing categories. 기존의 데이터를 이용하여 어떻게 새로운 데이터를 기존의 범주로 분류할지 결정하는 머신러닝 기술
- 예) 스팸메일 분류, 고객 이탈 여부 예측

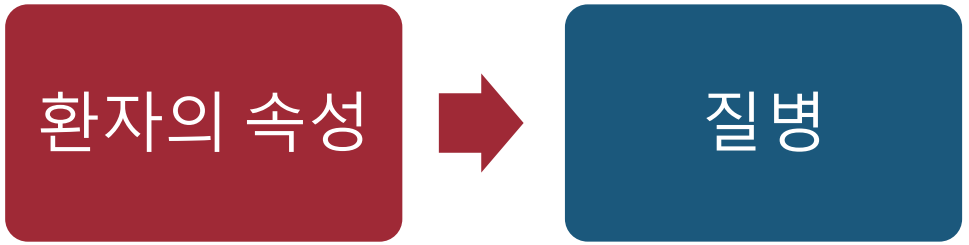


분류문제 (Classification Problems)

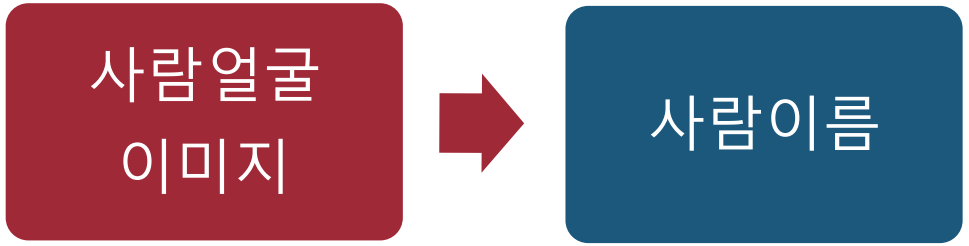
- Credit risk assessment
  크레딧 위험평가



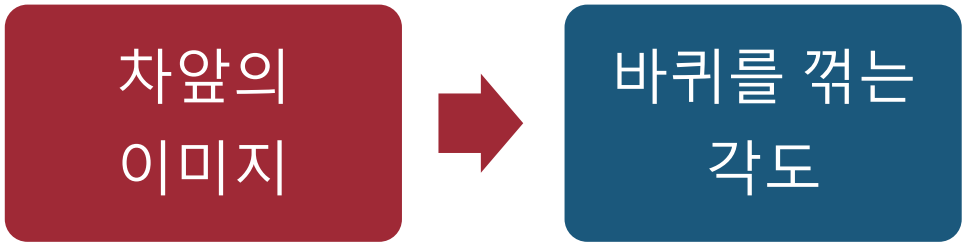
- Disease diagnosis
  질병진단



- Face recognition 얼굴인식



- Automatic steering
  자율조정장치



머신러닝을 위한 파이썬 라이브러리 (Python Libraries for Machine Learning)

Popular Python libraries

- NumPy 넘파이
- SciPy 싸이파이
- Pandas 팬다스
- SciKit-Learn 싸이킷런

Visualization libraries

- Matplotlib 맷플랏립
- Seaborn 씨본

분류 알고리즘 (Classification Algorithms)

Decision Tree
의사결정트리

Random Forest
랜덤포레스트

Linear Discriminant Analysis
선형판별분석

K-Nearest Neighbor K-
최근접이웃

Support Vector Machine
지원벡트머신

Neural Network
신경망

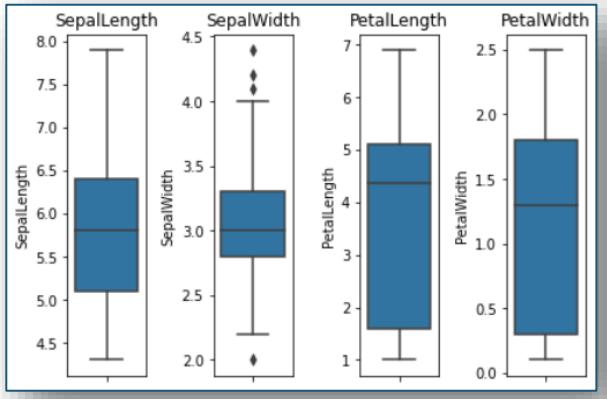
분류 모델 개발

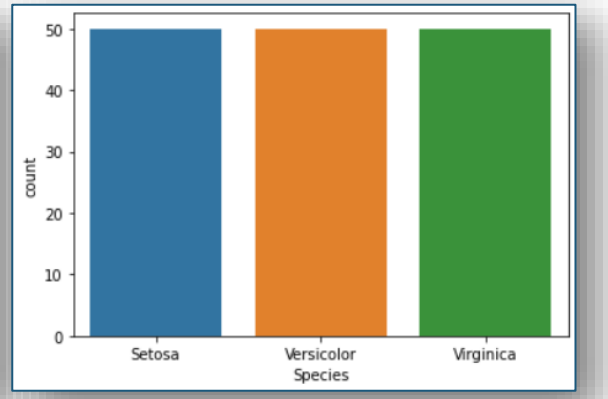


- Select the right machine learning algorithm to build a predictive model. Visualize the results to gain insights
  예측모델을 만들기 위해 적절한 기계학습 알고리즘을 택함. 인사이트를 도출하기 위해 시각화

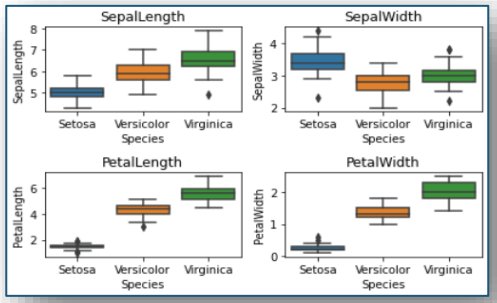
분류모델 EDA (계량형 → 범주형)

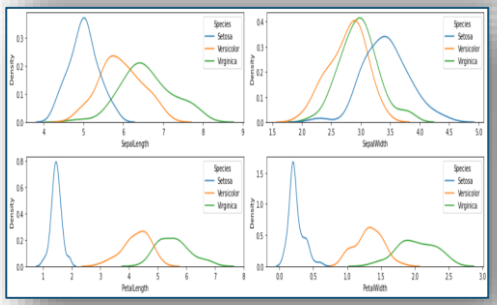
- Univariate analysis 일변량 분석
  - Descriptive statistics 기술통계
  - Frequency table 빈도테이블,
  - Histogram, box plot, bar plot 히스토그램, 박스플랏, 막대그래프

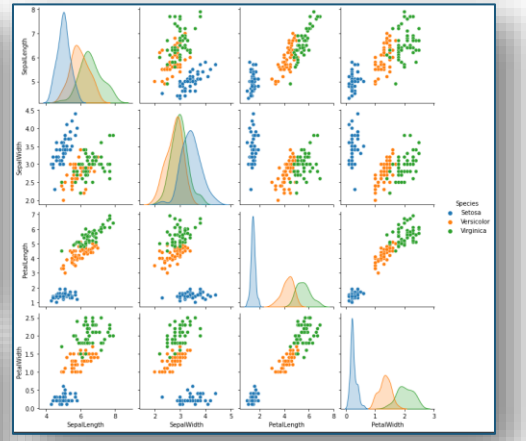




- Multivariate analysis 다변량 분석
  - Box plot, density plot 박스플랏, 밀도그래프
  - Scatter plot 산점도

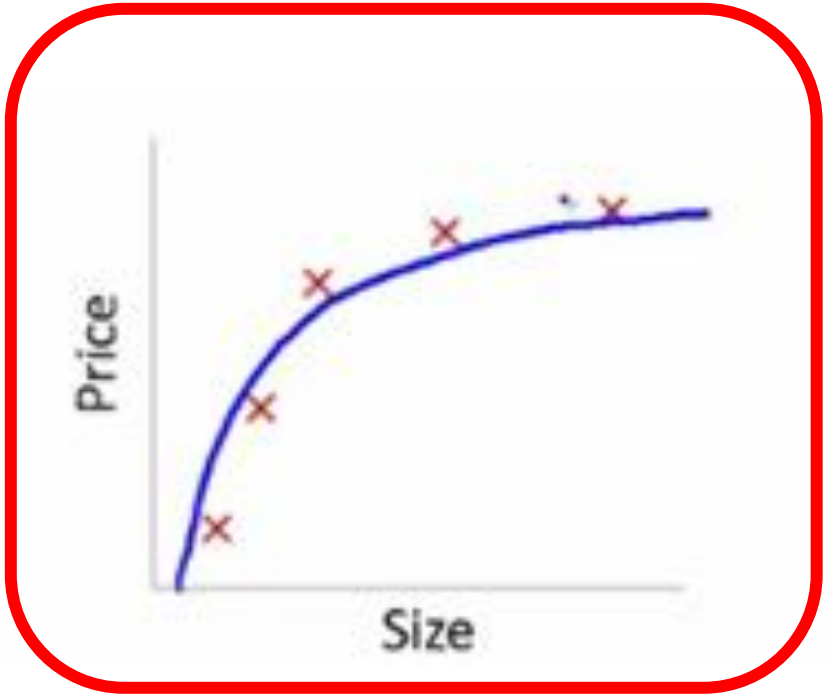


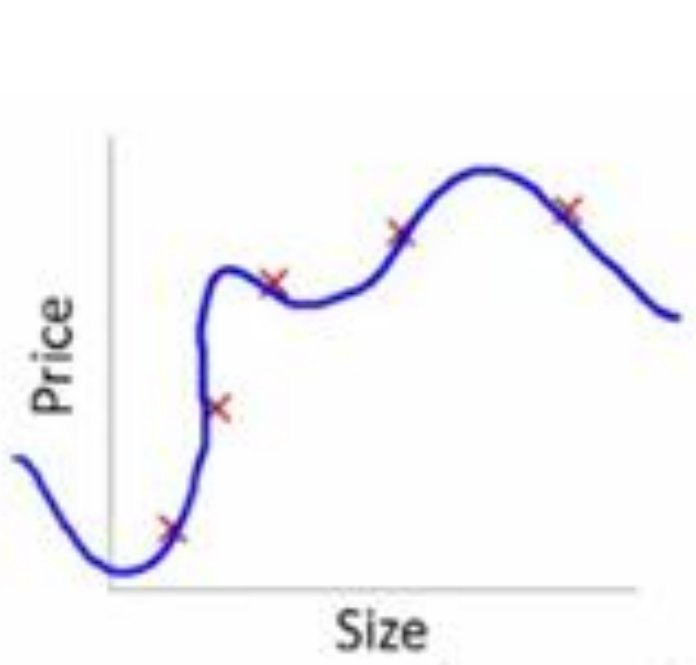




Fit of Models 모형적합







- Underfitting - high error from training data points.
훈련데이터에서 오차가 큼

- Overfitting - almost zero training error, but too sensitive and capture random patterns present only in the current data set. 훈련데이터에서는 오차가 거의 없음. 현재 데이터에 너무 민감하고, 무작위패턴까지 반영

Cross Validation 교차검증

- A techniques to build robust models which are not prone to overfitting. 오버피팅이 되는 것을 방지하고 견고한 모델을 개발하기 위한 기술
- Iterate over various models to find a better performing model. 더 성과가 좋은 모델을 찾기 위해 다양한 모델들을 반복함

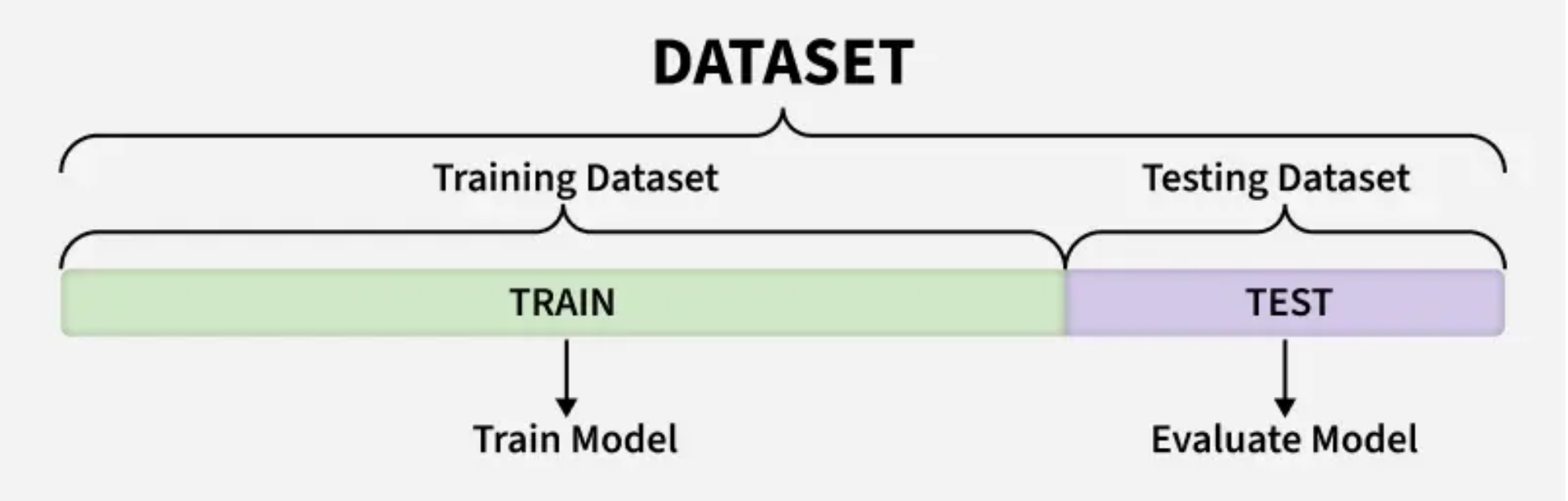
Holdout method

K-fold cross validation

Leave one out cross validation (LOOCV)

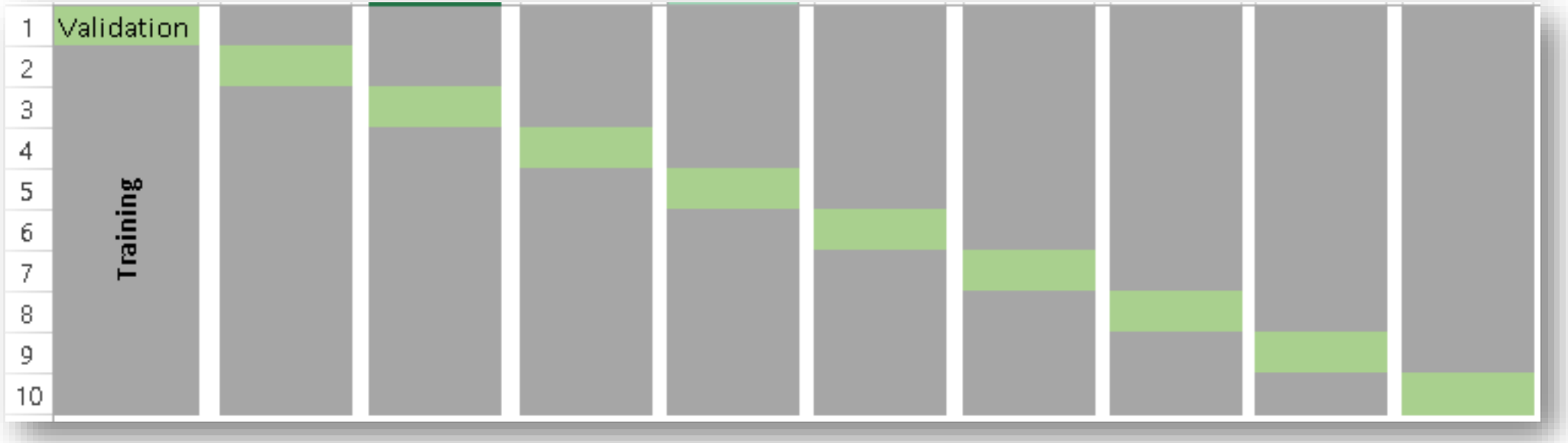
홀드아웃방법 (Holdout Method)

- The simplest kind of cross validation. 가장 간단한 교차검증
- The dataset is divided into two groups: training set and test set. 데이터를 훈련셋과 테스트셋으로 나눔
- Training set is used to develop a prediction model and testing set to evaluate the model. 훈련셋은 모델을 개발하기 위해 사용하고, 테스팅셋은 모델을 평가하기 위해 사용



k겹 교차검증 (k-fold Cross Validation)

- The data set is divided into k subsets. One of the k subsets is used as the test set and the other k-1 subsets are put together to form a training set.
  데이터셋을 k개로 나눈뒤 1개는 테스트셋으로 나머지는 트레이닝셋으로 사용



리브원아웃 교차검증 (Leave One Out Cross Validation)

- The extreme case of k-fold cross validation. k폴드 교차검증의 극단적인 케이스
- k equal to n, the number of data points in the set. k가 데이터의 갯수와 같음

Algorithm Comparison
알고리즘 비교 (평균, 표준편차)

- Compare the models to each other and select the most accurate. 모델을 비교하여 가장 정확한 것을 선택

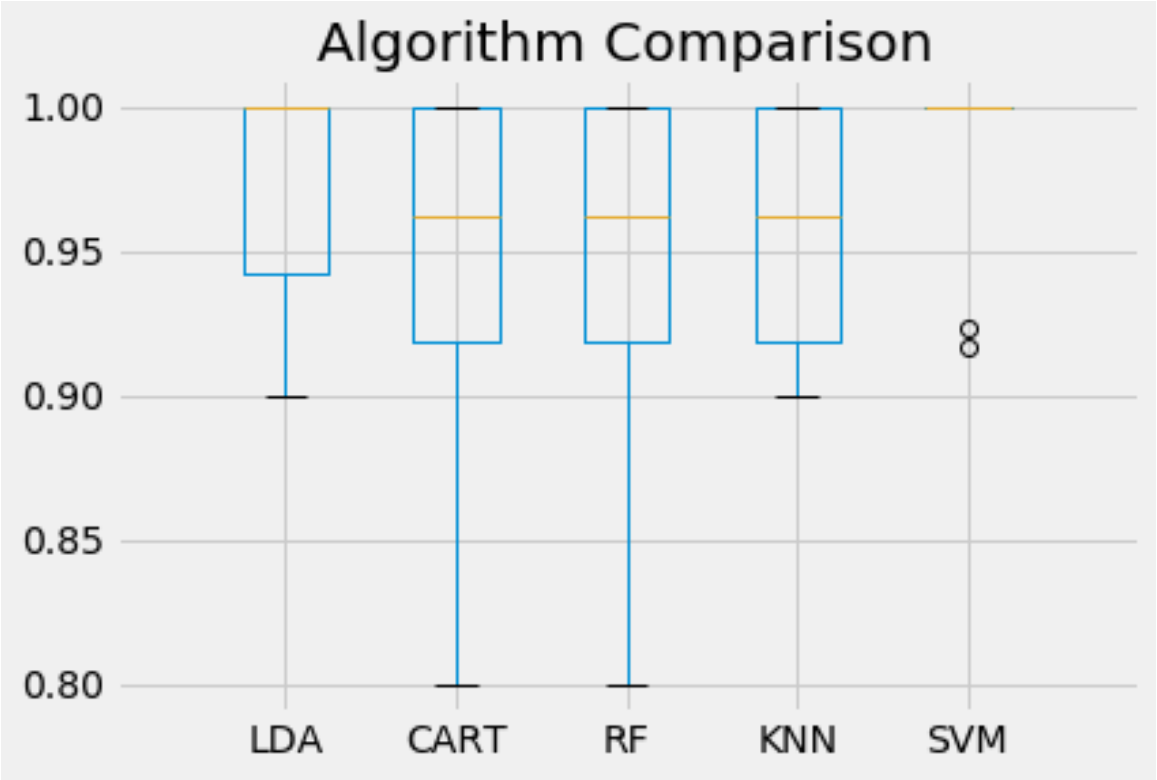
Model Accuracy Estimation

Model	CV Mean	CV Standdard Deviation
lda	0.973974358974359	0.04011017863311351
dtree	0.9381002331002332	0.07315791398631202
rf	0.9471911421911422	0.06257433351640862
knn	0.957191142191142 1	0.04326342995186461
svm	0.9839743589743589	0.03208331732369794 4

Algorithm Comparison
알고리즘 비교 (박스플랏)

- Many samples achieving 100% accuracy. 거의 100%의 정확도를 가짐

```
plt.boxplot([lda_cv,
dt_cv, rf_cv, knn_cv,
svm_cv])
plt.show()
```



Lab #4 Iris Example 붓꽃 예제

- Load the iris dataset and create a data frame named df (including target and feature names). 붓꽃데이터를 열어 타겟과 속성이름을 포함한 df 만들기
- X and y split. 독립, 종속변수 분리
- Splitting Dataset (test size=.2). 데이터셋 분리

Iris Dataset
붓꽃데이터

```
from sklearn import datasets
iris = datasets.load_iris()
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa

데이터 프레임 생성

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
# 데이터셋 생성
df = pd.DataFrame(iris.data, columns=['sepal_length',
'sepal_width', 'petal_length','petal_width'])
df['species'] = iris.target
df.head()
```

IV and DV Split
독립변수, 종속변수 분리

```
x = df.drop('species', axis=1)
y = df['species']
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Iris Dataset 붓꽃데이터

Splitting Dataset 데이터셋 분리

- Split the dataset into two, 80% of which we will use to train the models and 20% that we will hold back as a test dataset. 80프로는 모델을 훈련하기 위해 사용하고 나머지는 테스트 데이터로 사용

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test =
train_test_split(x,y,test_size=.2, random_state=1)
```

Random Number Seed 난수

- Reset the random number before building models to ensure the results are directly comparable. 똑같은 결과로 비교하기 위해 모델을 만들기 전 난수를 고정함

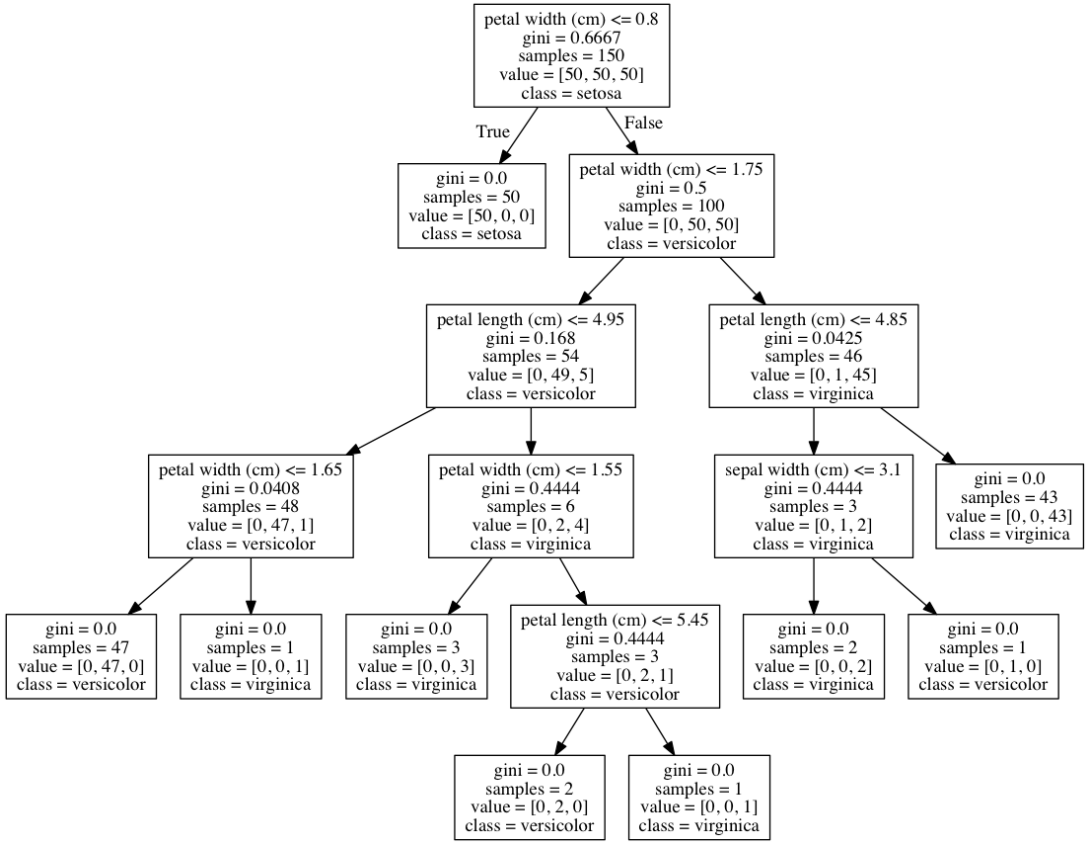
```
random_state=1
```

Lab #4 Iris Example 붓꽃예제

- Decision Tree 의사결정트리
  - Model evaluation (accuracy score) 모델평가
  - 5 fold cross validation (5 fold) 교차검증
  - Decision tree visualization (max depth=3) 의사결정트리 시각화

Decision Tree 의사결정트리

- A type of supervised learning algorithm which is mostly used in classification problems. 분류문제에서 가장 많이 쓰이는 지도학습 방법
- Split the sample into two or more homogeneous sub-groups based on most significant splitter in input variables.변수들을 가장 중요한 분류기준을 바탕으로 비슷한 그룹으로 나눔



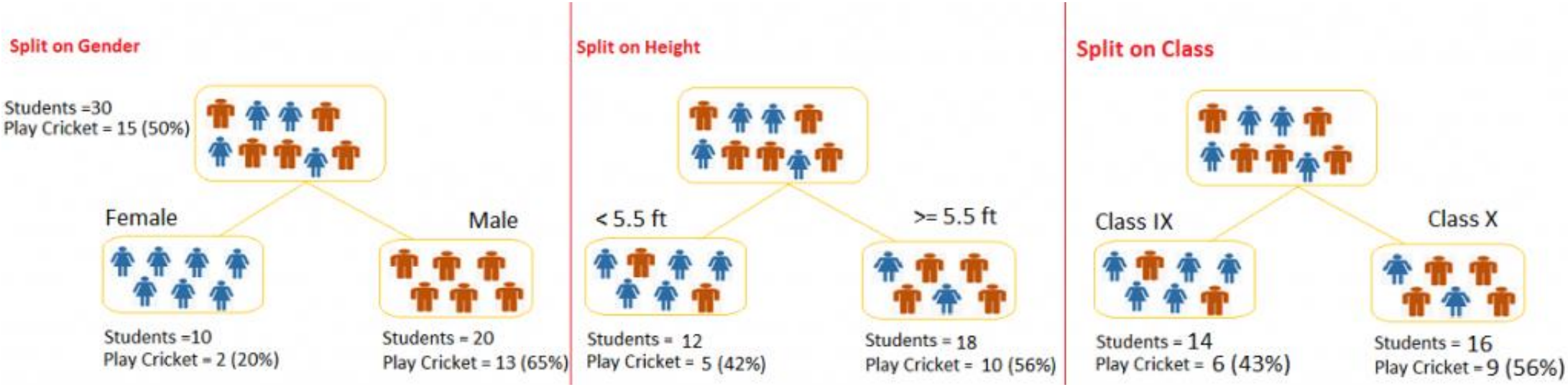
https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

Cricket Example 크리켓 예제

There are 30 students with three variables (gender, class, height). 15 out of 30 students play cricket in leisure time. Create a model to predict who will play cricket during leisure time. 30명의 학생을 구분지을 수 있는 세가지 변수 (성별, 반, 키)가 있습니다. 30명 중 15명은 여가시간에 크리켓을 합니다. 여가시간 동안 크리켓을 할 사람이 누군지 예측하는 모델을 개발하십시오

Solutions 해답

- Variable gender is able to identify best homogeneous sets compared to the other two variables. 다른 두 변수에 비해 성별이 그룹을 가장 잘 나눌 수 있음



Decision Tree Code

```
from sklearn.tree import DecisionTreeClassifier
dtc = DecisionTreeClassifier(random_state=0)
dtc.fit(x_train, y_train) #학습 데이터로 학습
```

You can also import DecisionTreeRegressor from sklearn.tree if you want to use a decision tree to predict a numerical target variable. 연속변수에 대한 예측인 경우는 리그레서를 사용

Model Evaluation for Decision Tree
모델 평가

```
dtc.score(x_test, y_test) #테스트 데이터의 정확도


from sklearn.metrics import accuracy_score
y_pred = dtc.predict(x_test) #테스트 데이터로 예측
dt_score = accuracy_score(y_test, y_pred)
print(dt_score)
```

Code for Cross Validation
교차검증 코드

```
from sklearn.model_selection import
cross_val_score

dt_cv = cross_val_score(dt, x_train, y_train,
cv=10, scoring='accuracy')

print(dt_cv)

print(dt_cv.mean(), lm_cv.std())
```

Decision Tree Visualization 시각화

```
from IPython.display import Image

from sklearn import tree

import pydotplus

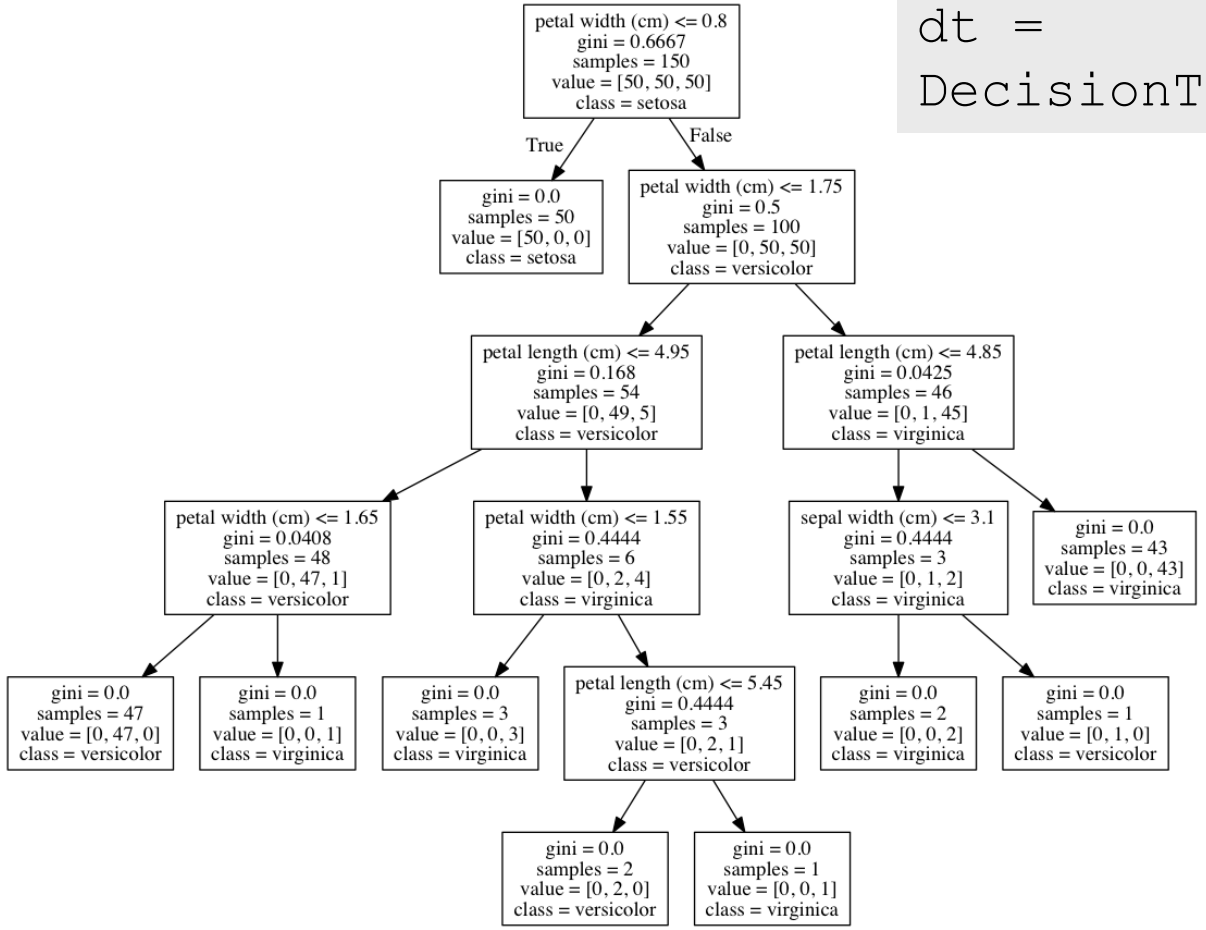
dot_data = tree.export_graphviz(dt,
out_file=None,feature_names=iris.feature_names,
class_names=iris.target_names)

graph = pydotplus.graph_from_dot_data(dot_data)

Image(graph.create_png())
```

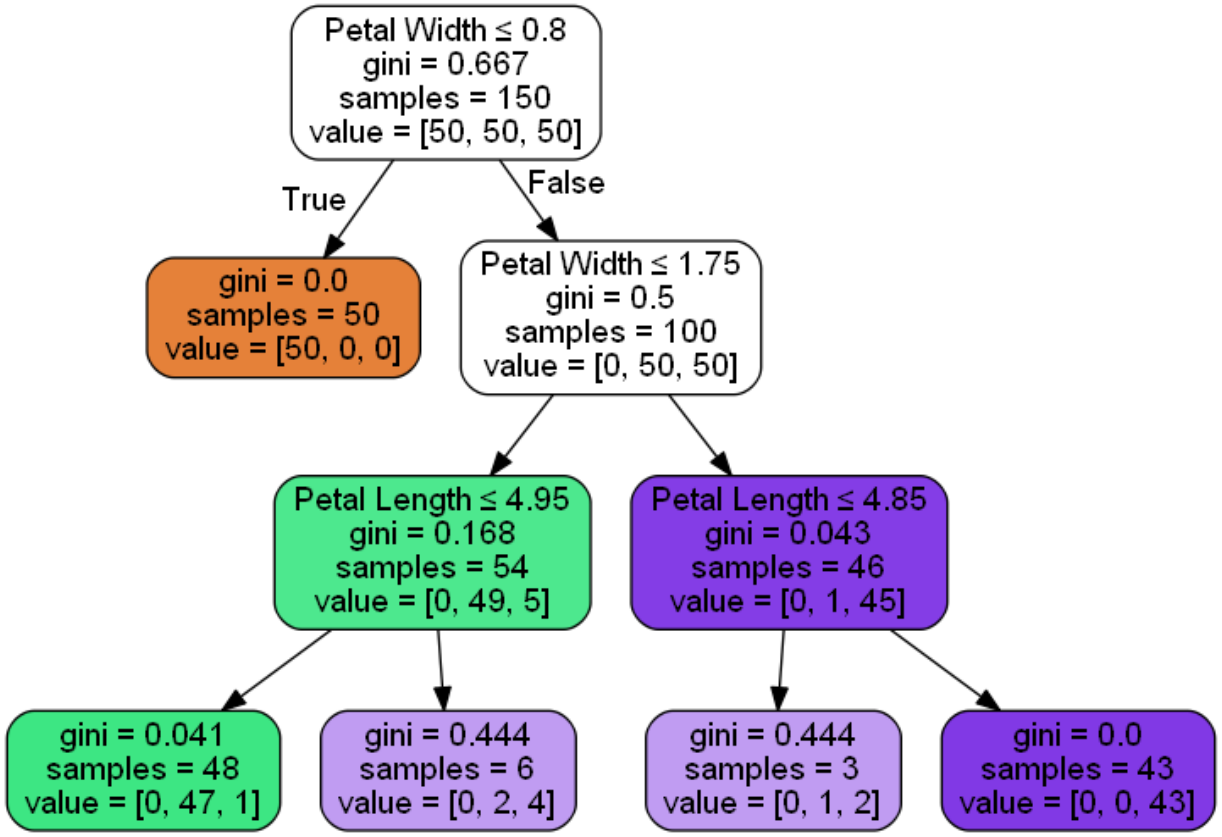
Graphviz is a tool for drawing graphics using dot files. Pydotplus is a module to Graphviz's Dot language.

Basic Decision Tree 기본 트리




```
dt =
DecisionTreeClassifier(random_state=0)
```

Decision Tree with Options




```
dt=DecisionTreeClassifier(max_depth=3,
random_state=0)
dt.fit(x_train, y_train)
data_feature_names = ['Sepal Lenth', 'Sepal
Width', 'Petal Length', 'Petal Width']
dot_data = tree.export_graphviz(dt,
feature_names=data_feature_names,
out_file=None, filled=True, rounded=True,
special_characters=True)
graph =
pydotplus.graph_from_dot_data(dot_data)
Image(graph.create_png())
```

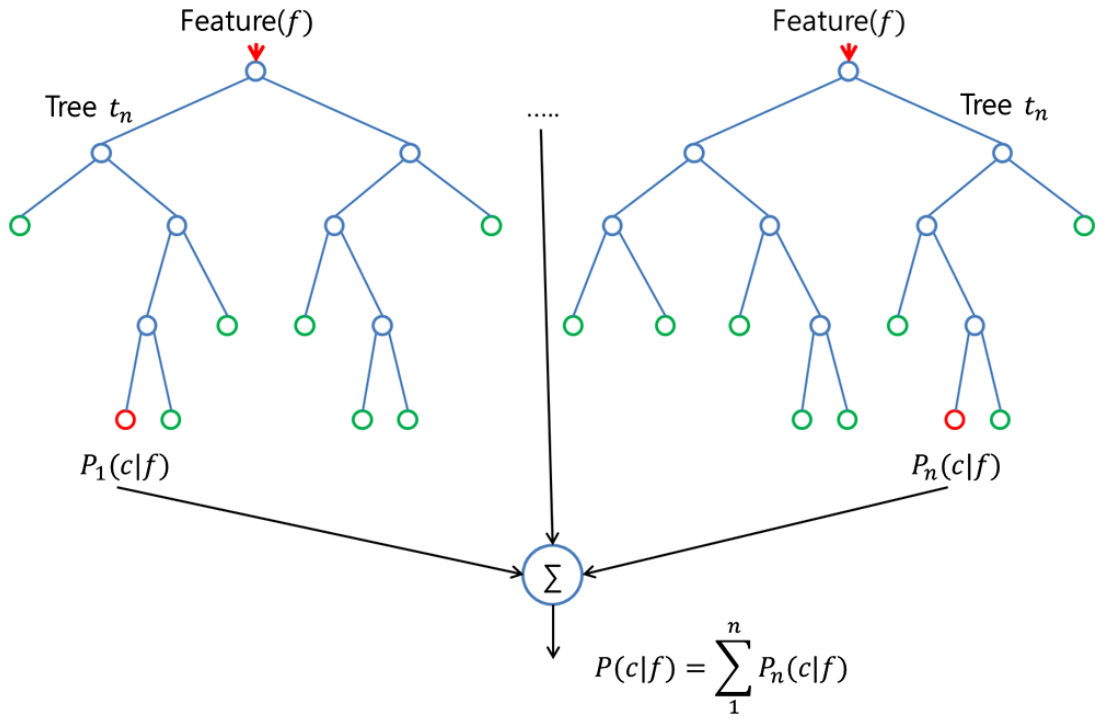
Save Image To File 이미지 저장

```
graph.write_pdf("iris.pdf") # Create PDF
graph.write_png("iris.png") # Create PNG
```

Lab #4 Iris Example 붓꽃예제

- Random Forest 랜덤포레스트
  - Model evaluation (accuracy score) 모델평가
  - 5 fold cross validation (5 fold) 5배 교차검증

Random Forest 랜덤포레스트



- A collection of classification tree.
  분류트리의 집합
- An ensemble tool which takes a subset of observations and a subset of variables to build a decision trees.
  관측치와 변수의 부분집합을 모아서 의사결정트리를 만드는 앙상블기법

sklearn.ensemble.RandomForestClassifier — scikit-learn 1.3.0 documentation

Code for Random Forest

- Builds multiple decision tree and amalgamate them together to get a more accurate and stable prediction. 여러 개의 의사 결정 트리를 만들고 이를 함께 결합하여 보다 정확하고 안정적인 예측을 함

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(random_state=0)
rf.fit(x_train, y_train)
```

n_estimators - number of trees to be grown in the forest. 트리의 갯수

Model Evaluation Random Forest
모델 평가

```
y_pred = rf.predict(x_test)

accuracy_score(y_test, y_pred)

cross_val_score(rf, x_train, y_train, cv=5,
scoring='accuracy')
```

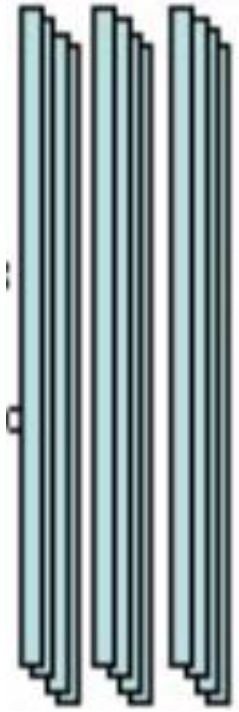
Linear Discriminant Analysis (LDA)
선형판별분석

- A well-known scheme for feature extraction and dimension reduction. 속성추출, 차원축소에 사용
- Widely used in high dimensional data such as face recognition and image retrieval. 얼굴인지, 이미지검색

https://scikit-learn.org/stable/modules/generated/sklearn.discriminant_analysis.LinearDiscriminantAnalysis.html

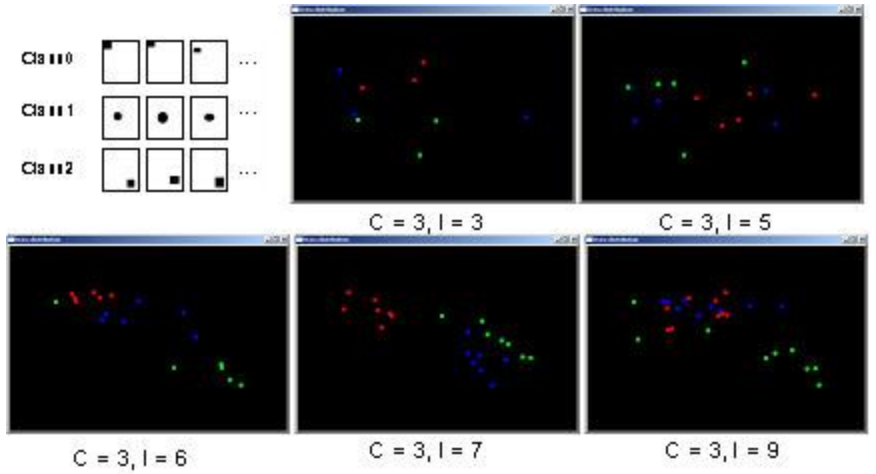
Training Set



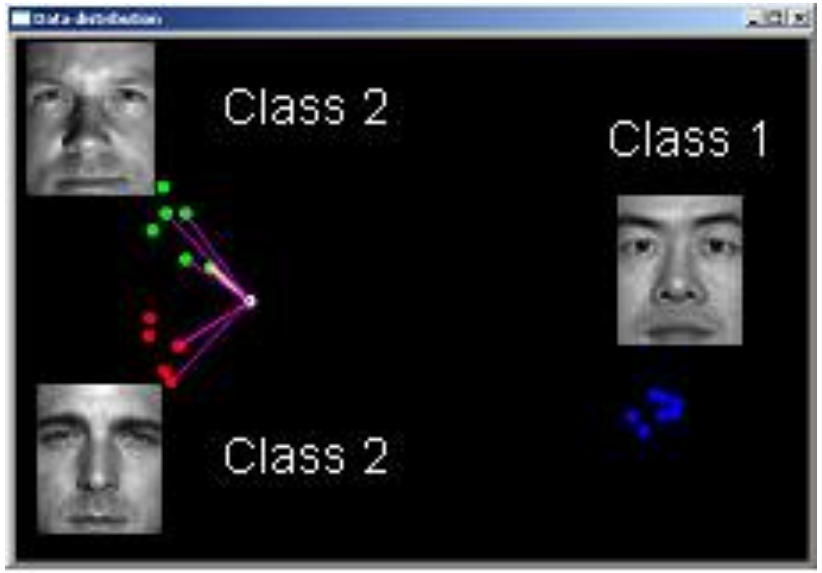


Dimension Reduction





3-Class/ 2-Dimension Map

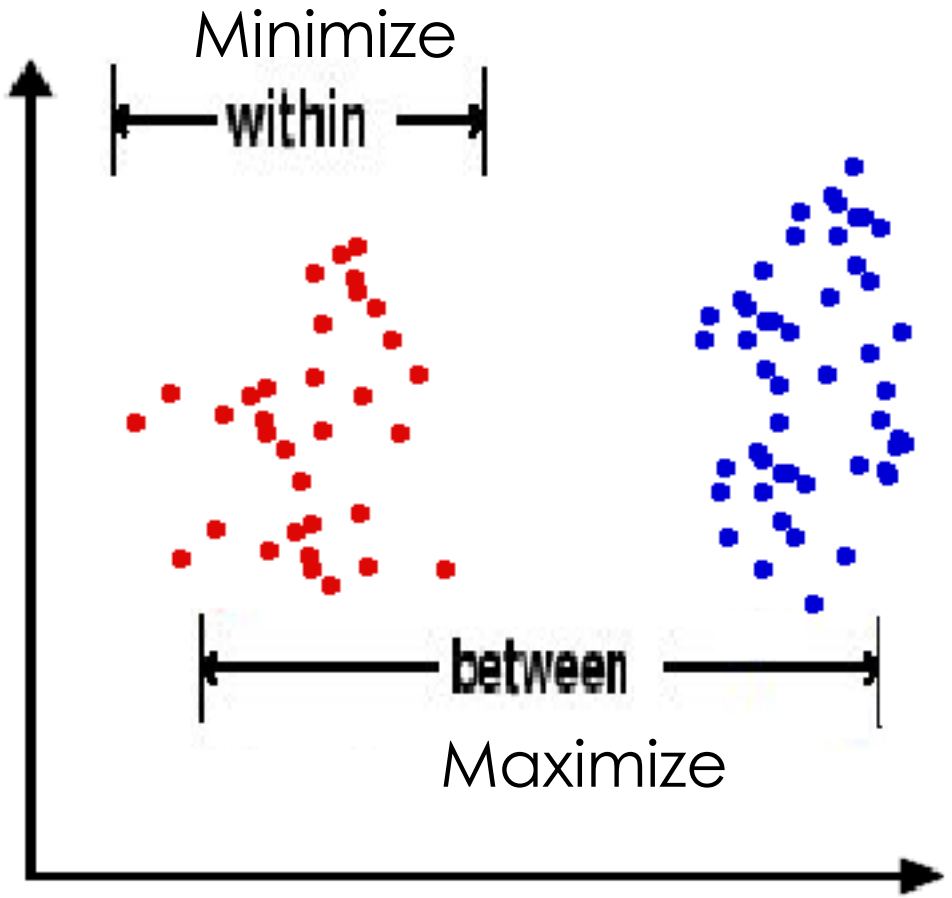


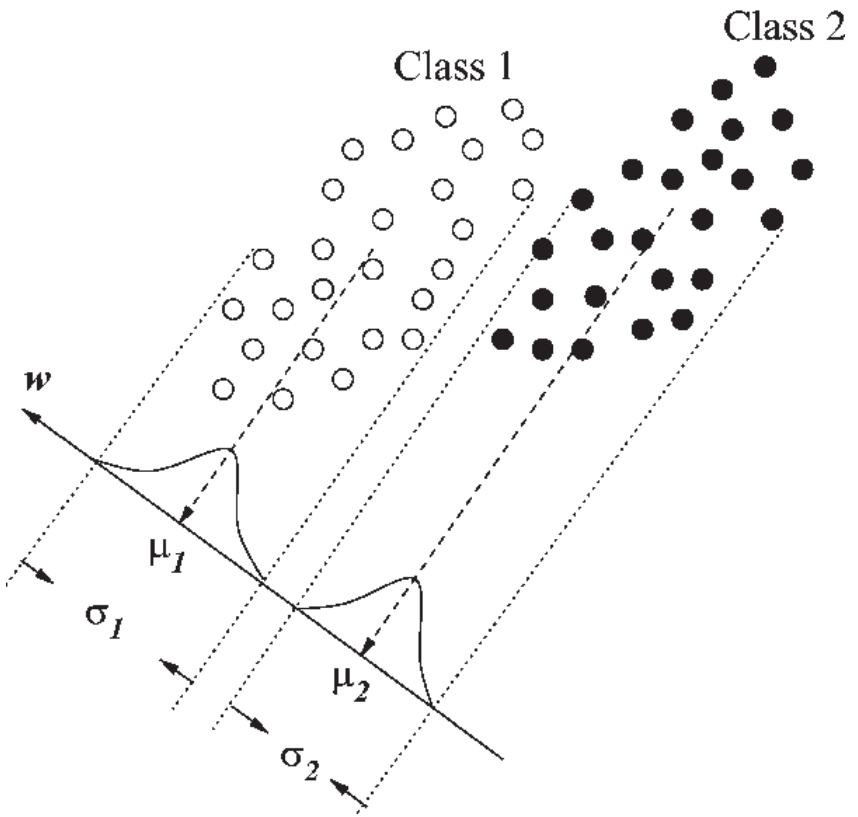
Testing Image



Applying Weighted K-Nearst Neighborhood

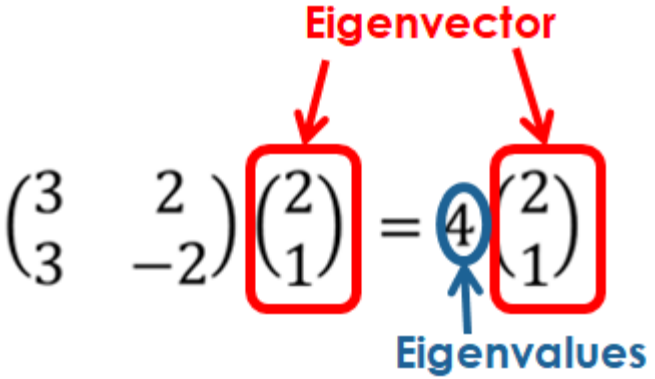
Dimension Reduction 차원축소





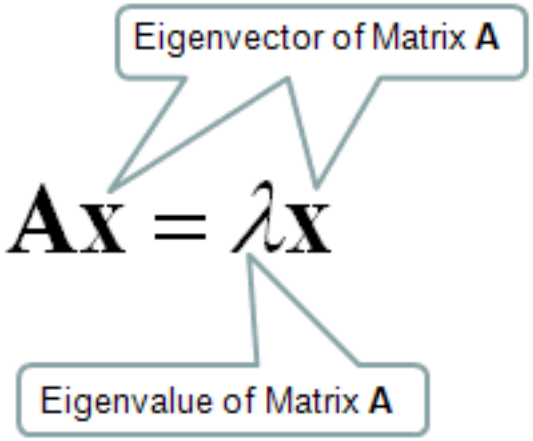
Eigenvalues and Eigenvectors
고유값, 고유벡터

- In linear algebra, an eigenvector or characteristic vector of a linear transformation is a non-zero vector that changes by only a scalar factor when that linear transformation is applied to it.
  숫자와 아이간벡터에 적용했을 때 원벡터에서 아이간벡터를 적용한 값이 나와서 아이간벡터를 이용하여 선형변형을 할 수 있음



$$\begin{pmatrix} 3 & 2 \\ 3 & -2 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = 4 \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} 8 \\ 4 \end{pmatrix} = \begin{pmatrix} 8 \\ 4 \end{pmatrix}$$



$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

LDA Algorithms 알고리즘

- Compute d-dimensional mean vectors for different classes from the dataset, where d is the dimension of feature space. 각 클래스로부터 속성의 차원을 가진 평균벡터들을 계산
- Compute in-between class and within class scatter matrices. 클래스 사이와 클래스 안에서의 스케터 메트릭스들을 계산
- Compute eigen vectors and corresponding eigen values for the scatter matrices. 아이간 벡터들과 스케터 메트릭스에 대응하는 아이간 밸류들을 계산
- Choose k eigen vectors corresponding to top k eigen values to form a transformation matrix of dimension d x k. 상위 k개의 아이간밸류들을 선택하여 d x k개를 가진 변환메트릭스 생성
- Transform the d-dimensional feature space X to k-dimensional feature space X_lda via the transformation matrix. 평균벡터를 변환메트릭스를 통해 변환

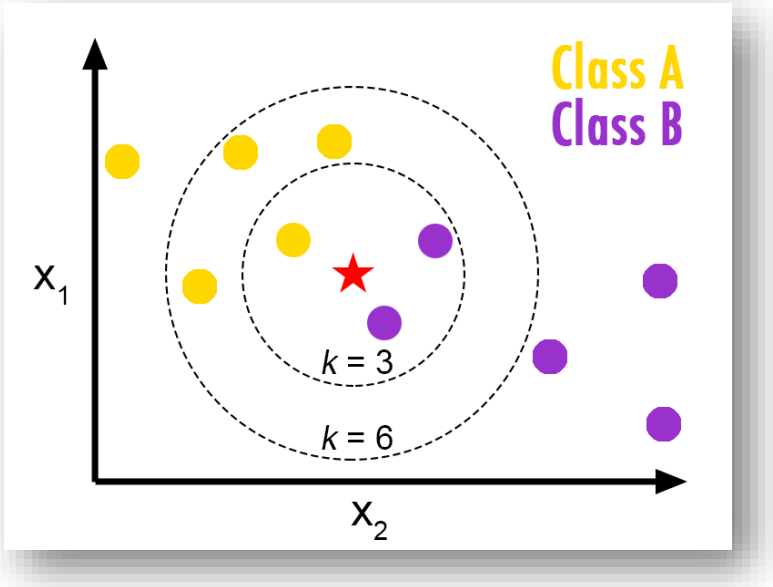
LDA Code

- Find a linear combination of features that separates two or more classes of object.s 두개 이상의 클래스들을 구별해주는 속성들의 조합

```
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis
lda = LinearDiscriminantAnalysis(n_components=1)
lda.fit(x_train, y_train)
lda.predict(x_test)
x_train = lda.transform(x_train, y_train)
x_test = lda.transform(x_test)
```

k-Nearest Neighbors (kNN)
최근접이웃법

- The case is assigned to the class that most common amongst its K nearest neighbors measured by a distance function. 거리에 따라 가까운 k개의 이웃들을 정하고 그 이웃들의 클래스를 따름
- The distance functions can be Euclidean, Manhattan, Minkowski (for continuous variables) and Hamming distance (for categorical variables). 거리함수들



https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

kNN Code 코드

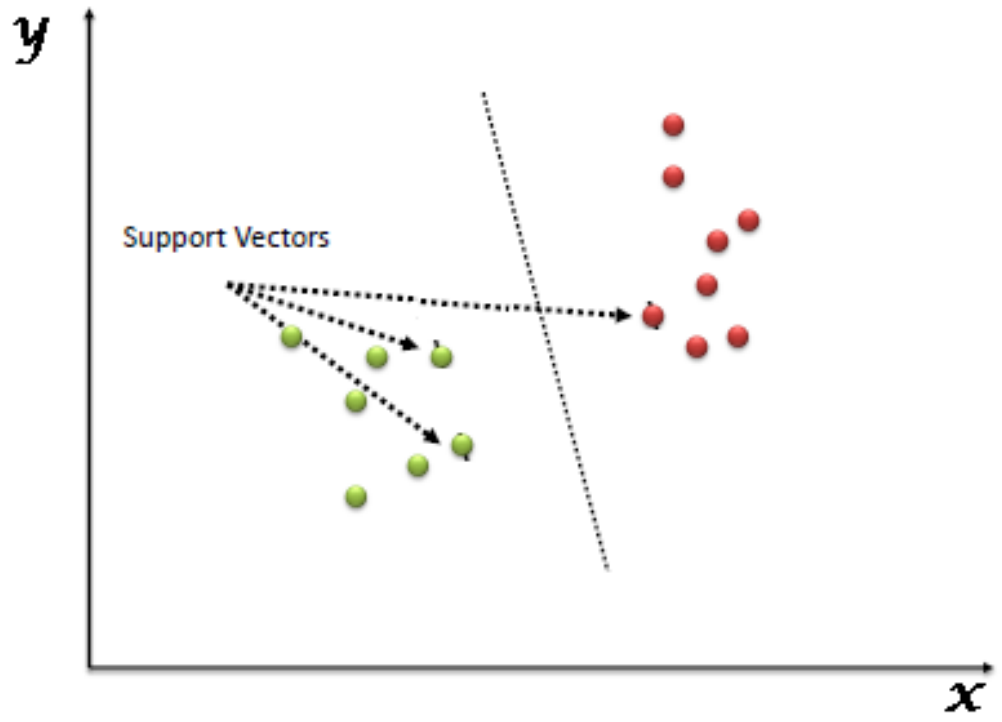
- Used for both classification and regression problems. 분류문제, 회귀문제에 다 사용됨

```
from sklearn.neighbors import
KNeighborsClassifier
knn = KNeighborsClassifier()
knn.fit(x_train, y_train) #학습
knn.evaluate(x_test, y_test) #평가
knn.predict(x_test) #예측
```

kNN Algorithm 알고리즘

- Load the data. 데이터를 가져옴
- Initialize the value of k. k값을 초기화 시킴
- For getting the predicted class, iterate from 1 to total number of training data points. 전체 데이터의 갯수만큼 계산함
  - Calculate the distance between test data and each row of training data. Here we will use Euclidean distance as our distance metric since it's the most popular method. The other metrics that can be used are Chebyshev, cosine, etc. 거리계산
  - Sort the calculated distances in ascending order based on distance values. 정렬
  - Get top k rows from the sorted array. 상위k개를 수집
  - Get the most frequent class of these rows. 가장 빈도가 높은 클래스를 정함
  - Return the predicted class. 정해진 클래스를 리턴

Support Vector Machines (SVM)



- Plot each data item as a point in n-dimensional space where n is number of features you have. 변수들의 공간 안에 모든 데이터 포인트들을 그림
- Then, perform classification by finding the hyper-plane that maximize the distances between nearest data point (margin). 가장 가까운 데이터 포인트들의 거리를 최대화시킬수 있는 하이퍼 플레인을 찾아 분류하는 방법

https://scikit-learn.org/stable/api/sklearn.svm.html

SVM Code

- A supervised machine learning algorithm which can be used for both classification or regression challenges. 분류와 회귀문제에 사용될 수 있는 지도학습 알고리즘

```
from sklearn.svm import SVC
svm = SVC()
svm.fit(x_train, y_train) #학습
svm.evaluate(x_test, y_test) #평가
svm.predict(x_test) #예측
```

Lab #4 Iris Example 붓꽃예제

- Create 5 machine learning models, pick the best and build confidence that the accuracy is reliable. 아이리스 데이터를 가지고 머신러닝 모델을 개발한 후 성능비교
  - Build DT, RF, LDA, kNN, and SVM models. 5개 모델 구축
  - Use 10-fold cross validation to compare the models. 교차검증을 사용하여 모델비교
  - Predict with actual data. 실데이터를 가지고 예측하기

여러 개의 모델을 한번에 실행하는 방법

```
models = [DecisionTreeClassifier(),
RandomForestClassifier(), LinearDiscriminantAnalysis(),
KNeighborsClassifier(), SVC()]
names = ['DT', 'RF', 'LDA', 'KNN', 'SVM']
for model, name in zip(models, names):
    model.fit(x_train,y_train) #훈련
    acc_score = model.score(x_test, y_test) #평가
    y_pred = model.predict(x_test) #예측
    sns.heatmap(confusion_matrix(y_test, y_pred),
annot=True, fmt='g') #혼동행렬
    plt.title('{}: {}'.format(name, acc_score))
    plt.show()
    print(classification_report(y_test, y_pred)) #분류보고서
```